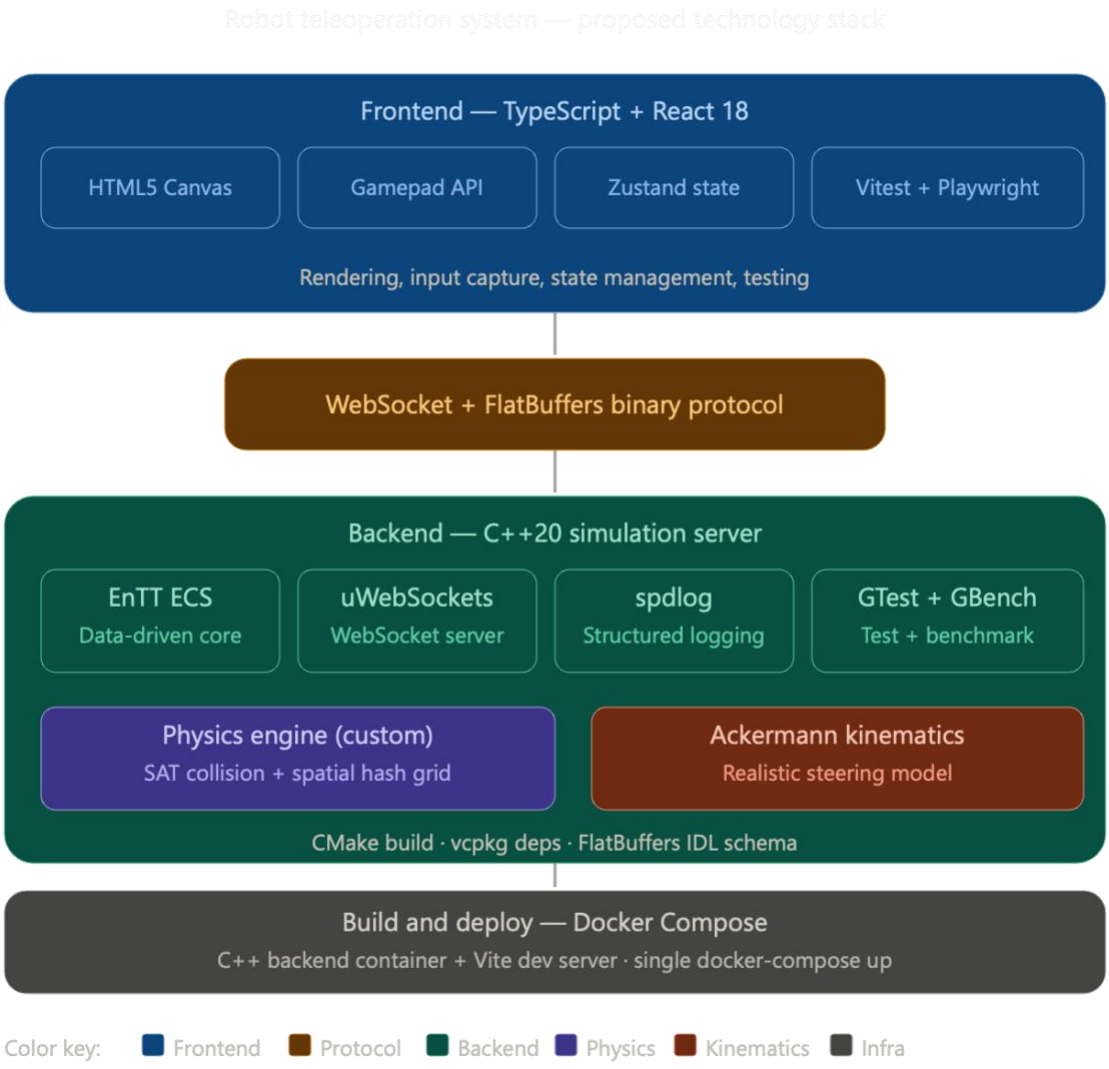

Here's the full technology stack proposal with justifications for each choice. Let me walk through each layer.

Technology Stack Proposal: Robot Teleoperation Simulator

1. Backend: C++20

Why C++20 specifically:

The simulation must handle 1000+ polygon objects with collision detection at 10Hz+ while maintaining deterministic, real-time performance. C++20 is the natural choice for several reasons. First, the `std::jthread` and `<stop_token>` primitives give clean lifecycle management

for the simulation loop without third-party threading libraries. Second, `concepts` and `constexpr` let us enforce SOLID interface contracts at compile time (e.g., a `Collidable` concept that constrains what enters the collision pipeline). Third, `std::span` and structured bindings make the data-driven ECS layout ergonomic without sacrificing cache locality. Finally, this is the lingua franca of the robotics industry — directly relevant to the target domain.

Key Backend Libraries:

Library	Purpose	Why This Over Alternatives
EnTT (ECS)	Entity-Component-System for all simulation entities	Cache-friendly SoA layout, header-only, perfect for Data-Driven Design — components are pure data structs, systems are functions over views. Beats hand-rolling an ECS.
uWebSockets	WebSocket server	Fastest C++ WebSocket library (benchmarks at 150K+ msg/sec). Single-threaded event loop avoids lock contention with the sim thread.
FlatBuffers	Binary serialization for state broadcasts	Zero-copy deserialization on the client, schema-defined IDL enforces protocol contracts. ~10x faster than JSON for high-frequency 60Hz state updates.
spdlog	Structured async logging	Lock-free async backend, supports structured JSON output for production log aggregation, compile-time log level stripping for release builds.
nlohmann/json	JSON for debug/config messages	Used only for low-frequency control commands and config; human-readable for debugging.
GoogleTest + GoogleBenchmark	Unit testing + benchmark testing	Industry standard, excellent parameterized testing support, benchmark fixtures for physics hot-path profiling.
RapidCheck	Property-based / hypothesis testing	QuickCheck-style generative testing for C++; generates random polygon configurations to stress-test collision detection invariants.
CMake + vcpkg	Build system + package management	CMake is the C++ standard; vcpkg provides reproducible dependency resolution with manifest mode.

2. Frontend: TypeScript + React 18

Why TypeScript + React:

TypeScript provides compile-time safety for the WebSocket protocol types (shared FlatBuffers-generated types ensure frontend/backend schema agreement). React 18's concurrent rendering ensures the UI stays responsive even during heavy Canvas redraws. The component model maps

cleanly to SOLID — each UI concern (Canvas renderer, input handler, HUD overlay, connection manager) is an isolated component with a single responsibility.

Key Frontend Libraries:

Library	Purpose	Justification
HTML5 Canvas API	2D polygon rendering	Native browser API, no dependency overhead. At 1000 polygons, Canvas outperforms SVG (DOM node overhead) and doesn't need WebGL complexity.
Gamepad API	Joystick/controller input	Native browser API, zero dependencies. Provides analog stick axes + button states at requestAnimationFrame rate.
Zustand	Client state management	Minimal, no boilerplate Redux overhead. Selector-based subscriptions prevent unnecessary re-renders — critical when state updates arrive at 30-60Hz.
Vite	Build tooling + dev server	Sub-second HMR, native ESM, built-in WebSocket proxy for development.
Vitest	Unit + integration testing	Vite-native, same config, fast execution. Compatible with React Testing Library.
Playwright	E2E testing	Cross-browser, supports Gamepad API mocking, can test WebSocket flows end-to-end.
flatbuffers (npm)	Binary protocol decoding	Generated TypeScript types from the same `.fbs` IDL schema used by the C++ backend — single source of truth.

3. Communication Protocol: WebSocket + FlatBuffers

Why this hybrid approach:

The system has two distinct message patterns with very different performance requirements:

High-frequency (30-60Hz) — State Broadcasts (Server → Client): These carry every entity's position, rotation, and polygon vertices. At 1000 entities, a JSON payload would be ~200KB per frame; FlatBuffers binary encoding reduces this to ~20KB with zero-copy deserialization on the client. This is the critical path.

Low-frequency (~10Hz) — Command Messages (Client → Server): Keyboard/gamepad inputs are small (velocity + steering angle). JSON is fine here — it's human-readable for debug logging and the payload is <100 bytes.

WebSocket (not WebRTC or raw TCP) because it works through firewalls/proxies, runs natively in every browser, and provides the bidirectional framing we need without HTTP polling overhead.

4. Physics Engine: Custom (SAT + Spatial Hash Grid)

Why custom instead of Box2D / Bullet:

The requirements are deliberately narrow — 2D convex polygons on a flat plane with simple collision response (velocity drops to zero on impact). Box2D is a full rigid-body solver with joints, friction, restitution, and continuous collision detection — massive overkill that would obscure the code structure the interviewer wants to evaluate.

A custom engine with SAT (Separating Axis Theorem) for narrow-phase and a spatial hash grid for broad-phase is:

- ~300 lines of focused, testable code
- Trivially benchmarkable (the hot path is isolated)
- Scales linearly to 1000s of objects with the grid ($O(n)$ broad phase vs $O(n^2)$ brute force)
- Demonstrates understanding of the underlying algorithms

5. Robot Kinematics: Ackermann Steering Model

Why Ackermann over simple differential drive:

The requirement says "velocity or steering angle" — this implies a car-like kinematic model, not a tank-drive. Ackermann steering with a bicycle approximation is the standard for wheeled mobile robots, produces realistic turning behavior, and is only ~20 lines of math. It also shows domain knowledge relevant to food robotics platforms.

6. Build & Deploy: Docker Compose

Why Docker Compose:

The deliverable must "turn over" — someone should be able to clone and run it. Docker Compose with two services (C++ backend built via multi-stage Dockerfile, Vite frontend dev server) gives a single `docker-compose up` command that works on any machine. No "install these 5 things first" instructions.

7. Testing Strategy Matrix

Test Type	Backend Tool	Frontend Tool	What It Validates
Unit	GoogleTest	Vitest	Individual functions: collision math, kinematic step, protocol encoding
Property/Hypothesis	RapidCheck	fast-check	"For any random polygon pair, SAT returns consistent results"
Benchmark	GoogleBenchmark	Lighthouse CI	Physics tick at 1000 objects < 5ms, Canvas render < 16ms
Integration	GoogleTest (fixtures)	Vitest + MSW	WebSocket handshake, state sync, command round-trip

Test Type	Backend Tool	Frontend Tool	What It Validates
E2E	—	Playwright	Full flow: connect → drive → collide → verify impact counter

8. Logging Strategy

Layer	Tool	Event Log	Debug Log
Backend	spdlog	Structured JSON: collisions, connections, tick timing	Verbose: entity state dumps, physics debug
Frontend	Custom logger	Structured: input events, state transitions, FPS drops	Console: WebSocket frames, render timing

Both layers use log levels (TRACE → CRITICAL) with compile-time stripping in release builds.

9. Key Design Decisions Summary

Decision	Choice	Tradeoff
Language	C++20	Higher implementation effort, but shows domain expertise and meets performance requirements
ECS over OOP	EnTT	Favors composition over inheritance, cache-friendly, aligns with Data-Driven Design
FlatBuffers over Protobuf	FlatBuffers	Zero-copy read (no deserialization step), better for real-time streaming
Custom physics over Box2D	SAT + grid	Simpler, testable, avoids unnecessary complexity
Canvas over WebGL	HTML5 Canvas	Sufficient for 2D polygons, no shader complexity
Zustand over Redux	Zustand	Less boilerplate, better performance for high-frequency updates

---